Snowflake Data Governance — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Snowflake Data Governance: Microsoft Purview 4
What you get . . . 4
What this delivers . . . 4
The deployment model . . . 5
Outcome . . . 5
1. Solution Overview — Snowflake Data Governance (Microsoft Purview) 5
What this solution is . . . 5
Why this matters . . . 6
How it works . . . 6
The governance design (used throughout) . . . 6
What you get . . . 7
2. Requirements Guide — Snowflake Data Governance (Microsoft Purview) 8
2.1 A Microsoft Purview account with Data Governance (Data Map/Catalog) enabled . . . 8
2.2 Least-privilege Purview data-governance roles . . . 8
2.3 Azure/Purview PowerShell + REST toolchain . . . 8
2.4 Network path to Snowflake and integration runtime . . . 8
2.5 A Snowflake scanning credential stored in Azure Key Vault (no secrets in code) . . . 9
2.6 The in-scope Snowflake databases and a pilot scope . . . 9
2.7 Classification scheme, data owners, and stakeholders (governance owned) . . . 9
Checklist . . . 9
Global Requirements — Microsoft Purview Solutions 10
1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10
2. Purview roles (least-privilege) and portal access . . . 11
3. Security & Compliance PowerShell + Graph toolchain . . . 11
4. Data sources & scope in scope . . . 11
5. Compliance, records, privacy & legal stakeholders . . . 11
6. A pilot scope and change control . . . 12
3. Implementation Guide — Snowflake Data Governance (Microsoft Purview) 12
Step 0 — Connect (all steps) . . . 13
Step 1 — Assess current state (Assess) . . . 13
Step 2 — Design the governance approach (Design) . . . 13
Step 3 — Prepare credential + register the source (Configure) . . . 13
Step 4 — Create and run the scan on the pilot (Pilot) . . . 14
Step 5 — Validate the pilot (Pilot → Validate) . . . 14
Step 6 — Expand and schedule (Expand) . . . 14

5. Testing Guide — Snowflake Data Governance (Microsoft Purview) 15
Test 1 — Prerequisites and enablement are in place . . . 15
Test 2 — Discovery inventory is complete . . . 15
Test 3 — Register-source dry-run makes no changes . . . 15
Test 4 — Source registered under the collection . . . 15
Test 5 — Credential is Key Vault-backed, no secret in code . . . 16
Test 6 — Scan created on the pilot scope . . . 16
Test 7 — Scan runs and completes on the pilot . . . 16
Test 8 — Assets appear in the catalog . . . 16
Test 9 — Classifications applied where expected . . . 16
Test 10 — Validation gate passes on the pilot . . . 16
Test 11 — Reporting evidence is produced . . . 16
6. Troubleshooting — Snowflake Data Governance (Microsoft Purview) 17
The scan fails to authenticate to Snowflake . . . 17
The scan fails to connect (network) . . . 17
`DGSNOW-Register-Source.ps1 -Commit` returns a REST error . . . 17
The scan runs but finds no assets . . . 18
Classifications aren't applied to columns I expected . . . 18
Scans are consuming more capacity/cost than expected . . . 18
`DGSNOW-Validate.ps1` shows the run status as failed or empty . . . 18
7. Best Practices — Snowflake Data Governance (Microsoft Purview) 19
8. Deliverables Summary — Snowflake Data Governance (Microsoft Purview) 20
Documentation . . . 20
Named artifacts . . . 20
PowerShell scripts . . . 21
What the buyer receives . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Snowflake Data Governance: Microsoft Purview

Snowflake is often the largest concentration of governed data in the estate — and the biggest blind spot in the data catalog. If Snowflake isn't in **Microsoft Purview**, you can't see what sensitive data it holds, can't classify it, and can't answer "where is our customer PII?" for the platform that has the most of it. This solution closes that gap by bringing Snowflake into the **Purview Data Map and Catalog** — registered, scanned, and classified — configured, piloted on a limited scope, validated, and defensible.

This is a complete, do-it-today package written for an administrator onboarding Snowflake to Purview for the first time. It delivers the capability end-to-end — from reading the Data Map's collections and integration runtimes, through preparing a **Key Vault-backed credential** (never a secret in a script), registering the source, and running the scan on **one pilot database**, to validating assets and classifications and only then expanding, scheduling, and curating — with every collection, source, scan, credential, and pilot scope named once and reused everywhere. Because Purview Data Governance has its own consumption model and the connector experience is **evolving**, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a step or a cost as permanent.

What you get

- A plain-language Solution Overview of Snowflake cataloging — register, scan, classify, curate — and how

it differs from DLP and protection.

- A Requirements Guide with verify / implement-if-missing for every prerequisite: a Purview account with

Data Governance, least-privilege collection roles, the Az/REST toolchain, the network path and integration runtime, a **Key Vault-backed** scanning credential (no secrets in code), the in-scope databases and pilot scope, and the classification scheme with data owners.

- An Implementation Guide of precise, ordered steps: assess, design the approach, prepare the credential and

register the source, create and run the scan on a **pilot database**, validate assets and classifications, then expand, schedule, and curate.

- Five PowerShell tools — Discovery, a prerequisite/health helper, a safe Registration/Scan tool (`-WhatIf`

by default; credential referenced by name from Key Vault), Validation, and Reporting — built on PowerShell/REST against Purview and Azure, **read-only or** `-WhatIf` **by default with no credentials in code**.

- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Snowflake governed from Purview: registered in the Data Map, scanned on a schedule, and its sensitive assets classified in the catalog — proven on a pilot database before scanning the account, with the

credential Key Vault-backed and no secret in code.

The deployment model

Assess → Design → Configure → **Pilot (one database)** → Validate → Expand — with a pilot gate before the account and scans scheduled only after validation.

Outcome

The Snowflake catalog blind spot closed — the largest data store visible, scanned, and classified in Purview — with the credential Key Vault-backed and the classification scheme, data ownership, and glossary left to the data-governance owners who make them.

1. Solution Overview — Snowflake Data Governance (Microsoft Purview)

What this solution is

Snowflake is often the largest concentration of governed data in the estate — and the biggest blind spot in the data catalog. If Snowflake isn't in **Microsoft Purview**, you can't see what sensitive data it holds, can't classify it, and can't answer "where is our customer PII?" for the platform that has the most of it. This solution closes that gap by bringing Snowflake into the **Purview Data Map and Data Catalog**:

- **Register** the Snowflake source in the Purview **Data Map** under a governance collection, so it's a first-class part of the catalog.
- **Scan** Snowflake — connecting with a securely-stored credential (never a secret in a script), through a **self-hosted or managed integration runtime** where network access requires it — to extract the technical metadata (databases, schemas, tables, columns).
- **Classify** the scanned assets against Purview's **classifications / sensitive information types** (credit-card, national ID, and your custom types) so sensitive columns are identified automatically.
- **Catalog and curate** the results — assets discoverable in the Purview catalog, with scope for glossary terms, data owners, and lineage — so the platform is governed, not just connected.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), the **Purview Data Map**, and PowerShell/REST against Purview and Azure — no third-party tooling. The source is registered, **scanned first on a limited pilot scope (one database/schema)**, validated against pass/fail gates, then expanded.

> ***This is data *governance* (Data Map/Catalog), not data *loss prevention*.*** *This solution catalogs and classifies Snowflake so you can *see and understand* the data. It does not enforce DLP on Snowflake or apply protection inside Snowflake — that's a different capability. Cataloging is the foundation that makes downstream governance decisions possible; this solution delivers that foundation.*

Why this matters

• **You can't govern what you can't see.** A data catalog that omits Snowflake is missing the platform most likely to hold regulated data. Registering and scanning it makes the largest data store visible.

• **Classification finds the sensitive columns for you.** Scanning against classifications/SITs surfaces which Snowflake columns look like PII, financial, or health data — at scale, instead of by interview.

• **The catalog answers the questions auditors ask.** "Where is customer PII, who owns it, and how is it classified?" becomes a catalog query once Snowflake is scanned and curated.

• **It is defensible and auditable.** "Snowflake is registered, scanned on a schedule, and its sensitive assets are classified in the catalog" is a statement you can evidence with the Data Map and a report, not a hope.

How it works

1. **Assess.** Read the Purview account, its **collections**, existing **registered sources** and scans, and confirm the network path to Snowflake and where a **self-hosted integration runtime (SHIR)** is required — all read-only.

1. **Design.** Decide the **collection** for Snowflake, the **credential** approach (a securely-stored secret in **Azure Key Vault**, referenced by Purview — never in a script), the **scan rule set** (which classifications apply), the **scope** (which databases/schemas), and the **pilot scope** (one database/schema). The classification scheme and data-owner assignments are stakeholder determinations.

1. **Configure.** Register the `Snowflake – Source` in the Data Map under the collection, create the `Snowflake – Scan` with its scan rule set and Key Vault-backed credential, scoped to the pilot.

1. **Pilot.** Run the scan on the **pilot scope first** (one database/schema) — never the whole account on day one. Scans consume resources and surface findings that need curation; prove the scope and results first.

1. **Validate.** Confirm the scan completed, assets appear in the catalog with the expected metadata, and classifications applied where expected, against the pass/fail gates.

1. **Expand.** Broaden the scan scope to the intended databases/schemas, put the scan on a **schedule**, curate ownership/glossary, and hand ongoing cataloging and reporting to the data-governance team.

The governance design (used throughout)

Names below are examples; the **classification scheme, data owners, and glossary** are stakeholder **determinations** owned by your data-governance function — this solution provides the technical registration/scan/classification.

Element	Design intent
Collection	`Snowflake – Collection` (under the Purview account root/domain)
Registered source	`Snowflake – Source` (the Snowflake account/server in the Data Map)
Scan	`Snowflake – Scan` (credential + scan rule set + scope + schedule)
Scan rule set	`Snowflake – Scan Rule Set` (which classifications/SITs apply)
Credential (Key Vault-backed)	`Snowflake – Scan Credential` — secret stored in Azure Key Vault, referenced by Purview
Pilot scope	`Snowflake – Pilot Scope – Stage1` — one database/schema
Design map	`Snowflake Governance – Map`
Coverage report	`Snowflake – Coverage Report`

> ***Licensing / plan note — verify against your own tenant.*** *Microsoft Purview* ***Data Governance*** *(the Data Map / Unified Catalog) has its own* ***consumption/capacity*** *and licensing model, distinct from the Microsoft 365 E5 compliance licensing used by DLP/labels — and the Data Governance experience is* ***actively evolving*** *(the classic Data Map and the newer unified catalog experience). Confirm the current Snowflake connector support, the exact registration/scan steps, integration-runtime requirements, and the pricing/consumption model for *your* Purview account in Microsoft's documentation before you design. Where a capability differs in your tenant, follow the current portal flow and record deviations in* `Snowflake Governance – Map`*.*

What you get

• This **Solution Overview**, a **Requirements Guide** (every prerequisite with verify / implement-if-missing), a precise **Implementation Guide**, a **Testing Guide** with pass/fail gates, a **Troubleshooting Guide**, and field-proven **Best Practices**.

• **Five PowerShell tools** — Discovery, a prerequisite/health helper, a safe Registration/Scan tool (`-WhatIf` by default; credential referenced from Key Vault, never a secret in code), Validation, and Reporting — built on PowerShell/REST against Purview and Azure, **read-only or `-WhatIf` by default and with no credentials in code**.

> ***Boundary.*** *The* ***classification scheme****, the* ***data owners/stewards****, the* ***glossary****, and any regulatory obligation on Snowflake data are* ***determinations owned by your data-governance, compliance, and privacy functions****. This solution provides the technical registration, scan, and classification — and never embeds a Snowflake secret — not the determination.*

2. Requirements Guide — Snowflake Data Governance (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 A Microsoft Purview account with Data Governance (Data Map/Catalog) enabled

Description: A **Microsoft Purview account** with the **Data Map / Data Catalog (Data Governance)** capability, and the associated **consumption/capacity** model understood — this is distinct from the M365 E5 compliance licensing used by DLP/labels. (Global Requirement 1.) **Why:** Registering, scanning, and cataloging Snowflake happens in the Purview Data Map, which has its own account/consumption model. **Verify:** In the Purview portal confirm the Data Map / catalog experience is available; `DGSNOW-Check-Prereqs.ps1` reports the Purview-account signal. **Verify the current licensing/consumption model and connector support for your account** — the Data Governance experience is evolving. **Implement:** Provision/enable the Purview account and its Data Governance capability; confirm the pricing model with the owner before running scans (scans consume capacity).

2.2 Least-privilege Purview data-governance roles

Description: Least-privilege **Data Map / catalog roles** — **Collection admin** (to manage the collection), **Data source admin** (to register sources and manage scans), and **Data curator** (to curate assets/classifications) — scoped to `Snowflake – Collection`. Reserve broad roles for one-time setup. (Global Requirement 2.) **Why:** The Data Map uses collection-scoped roles distinct from the compliance roles; registration/scan requires Data source admin, curation requires Data curator. **Verify:** In the Purview portal → the collection → **Role assignments** confirm the roles; `DGSNOW-Check-Prereqs.ps1` reports whether the signed-in identity can read the Data Map. **Implement:** Assign the least-privilege collection roles under change control before configuring.

2.3 Azure/Purview PowerShell + REST toolchain

Description: A workstation with **Az.Accounts** (and, where used, **Az.Purview**) for authenticating to Azure/Purview and calling the **Purview Data Map REST API** (`Invoke-RestMethod` with a bearer token from `Get-AzAccessToken`), with a modern-auth / MFA admin. (Global Requirement 3.) **Why:** Data Map registration/scan operations are driven through the Purview REST API and Az modules; the provided scripts use them read-only or with `-WhatIf` gating. **Verify:** `Get-Module -ListAvailable Az.Accounts`; `Connect-AzAccount` succeeds and a token for the Purview endpoint can be acquired. `DGSNOW-Check-Prereqs.ps1` reports module/connection state. **Implement:** `Install-Module Az.Accounts -Scope CurrentUser` (add `Az.Purview` if used); confirm modern-auth sign-in before running steps.

2.4 Network path to Snowflake and integration runtime

Description: A working **network path** from the scanning runtime to the Snowflake account, and — where Snowflake is not reachable from the Purview managed runtime — a **self-hosted integration runtime (SHIR)** installed and registered to the Purview account. (Global Requirement 4.) **Why:** The scan connects to Snowflake to read metadata; if the network requires it, a SHIR provides the reachable, secure connection. **Verify:** Confirm the runtime can reach the Snowflake host/port; `DGSNOW-Discover.ps1` reports registered integration runtimes. **Verify the current Snowflake connector's runtime requirements against Microsoft's docs. Implement:** Install/register a SHIR on a host with Snowflake connectivity where the managed runtime can't reach it; open the required network path under change control.

2.5 A Snowflake scanning credential stored in Azure Key Vault (no secrets in code)

Description: A **Snowflake login** (user/role with least-privilege read access to the in-scope databases' metadata) whose **secret is stored in Azure Key Vault**, with the Key Vault **linked to the Purview account** and the credential referenced by the scan — **never** a secret in a script or document. (Global Requirement 3 — no hardcoded secrets.) **Why:** The scan authenticates to Snowflake; Purview reads the secret from Key Vault at scan time so no credential is embedded anywhere. **Verify:** The Snowflake login exists with least-privilege read; the Key Vault holds the secret and is linked in the Purview account (Management → Credentials / Key Vault); `DGSNOW-Check-Prereqs.ps1` reminds you to confirm the Key Vault linkage. **Implement:** Create the least-privilege Snowflake login, store its secret in Key Vault, link the Key Vault to Purview, and create the Purview credential referencing it — before creating the scan.

2.6 The in-scope Snowflake databases and a pilot scope

Description: The **Snowflake databases/schemas in scope** for cataloging, and a **pilot scope** (`Snowflake – Pilot Scope – Stage1`: one database/schema) to validate on before scanning the account. (Global Requirements 4 & 6.) **Why:** Scans key off scope; a first full-account scan is expensive and surfaces more findings than you can curate. Prove on a bounded pilot first. **Verify:** The in-scope databases and the pilot scope are agreed with the data-governance/Snowflake owners; `DGSNOW-Discover.ps1` reports existing sources/scans. **Implement:** Enumerate the in-scope databases and define the pilot scope before creating the scan.

2.7 Classification scheme, data owners, and stakeholders (governance owned)

Description: The **classifications / sensitive information types** to apply (built-in + custom), the **data owners/stewards** and **glossary** approach, and engagement with **data-governance, compliance, and privacy** stakeholders. (Global Requirements 4 & 5.) **Why:** The scan rule set applies classifications; which classifications matter, who owns the data, and the glossary are **determinations** owned by data-governance, not technical defaults. **Verify:** The classification scheme and ownership approach are documented and approved; named stewards recorded in `Snowflake Governance – Map`. **Implement:** Confirm the classification scheme and ownership before curating; where incomplete, scan with the default rule set and defer curation determinations to the owners.

Checklist

• Microsoft Purview account with Data Governance (Data Map/Catalog) available; consumption model confirmed

• Least-privilege collection roles (Collection admin / Data source admin / Data curator) on `Snowflake – Collection`

• Az.Accounts (+ Az.Purview where used) installed; `Connect-AzAccount` + Purview token succeed

• Network path to Snowflake confirmed; SHIR installed/registered where required

• Least-privilege Snowflake login; secret in **Azure Key Vault** linked to Purview (no secrets in code)

• In-scope Snowflake databases enumerated; `Snowflake – Pilot Scope – Stage1` (one db/schema) defined

• Classification scheme + data owners/glossary approach agreed; stewards named

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Snowflake Data Governance (Microsoft Purview)

Follow these steps in order. The model is **Assess** → **Design** → **Configure** → **Pilot (limited scope)** → **Validate** → **Expand**. Every write is **piloted on** `Snowflake – Pilot Scope – Stage1` **(one database/schema) first**, previewed with `-WhatIf`, and reversible. Named objects are introduced once and reused everywhere:

Object	Name
Collection	`Snowflake – Collection`
Registered source	`Snowflake – Source`
Scan	`Snowflake – Scan`
Scan rule set	`Snowflake – Scan Rule Set`
Credential (Key Vault-backed)	`Snowflake – Scan Credential`
Pilot scope	`Snowflake – Pilot Scope – Stage1`
Design map	`Snowflake Governance – Map`
Coverage report	`Snowflake – Coverage Report`

> ***No secret ever lives in a script.*** *The Snowflake credential's secret is stored in **Azure Key Vault**, the Key Vault is **linked to the Purview account**, and the Purview **credential** references it by name. The scripts pass the *credential name* — never a password.* ***Verify the current Snowflake connector flow and integration-runtime requirements against your Purview account.***

Step 0 — Connect (all steps)

Open PowerShell 5.1+ or 7+ and connect with a modern-auth/MFA admin:

```
Connect-AzAccount                                  # Azure sign-in (for the Purview/Key Vault context)
# Data Map REST calls use a bearer token: (Get-AzAccessToken -ResourceUrl "https://purview.azure.net").Tok
```

Keep credentials out of scripts — sign in interactively; the scripts acquire a token at runtime. **Pass/ fail:** sign-in succeeds and a token for the Purview endpoint can be acquired.

Step 1 — Assess current state (Assess)

1. Run `DGSNOW-Discover.ps1 -PurviewAccount "<your-purview-account>"`. Read-only, it reports the Data Map

collections, existing **registered sources** and **scans**, and any registered **integration runtimes** — so you know what exists before adding Snowflake.

1. Run `DGSNOW-Check-Prereqs.ps1` — modules/connection, the Purview-account and Data-Map read signals, and

reminders to confirm the Key Vault linkage, SHIR, and roles — PASS/GAP per signal.

1. Record every GAP and remediate before configuring. **Pass/fail:** the inventory and prereq report

complete; existing collections/runtimes captured in `Snowflake Governance – Map`.

Step 2 — Design the governance approach (Design)

1. With data-governance, decide the **collection** (`Snowflake – Collection`), the **classifications /

SITs **the scan rule set applies, the** scope **(which databases/schemas), and the** pilot scope** (one database/schema).

1. Decide the **credential** approach: create the least-privilege **Snowflake login**, store its secret in

Azure Key Vault, and plan the Purview **credential** (`Snowflake – Scan Credential`) that references it. Confirm whether a **SHIR** is required for network reachability.

1. Record everything (collection, scope, rule set, credential plan, SHIR, stewards) in `Snowflake

Governance – Map`. **Pass/fail:** the map is agreed and signed off; the classification scheme and data owners are owned by stakeholders, not IT.

Step 3 — Prepare credential + register the source (Configure)

Do this **for the pilot first** — one database/schema.

1. **Prepare the Key Vault-backed credential (portal + Azure).** In Azure, store the Snowflake secret in

Key Vault; in the Purview portal **Management** → **Credentials** link the Key Vault and create `Snowflake – Scan Credential` referencing the secret. **No secret is entered in any script.**

1. **Register the source.** Preview then commit:

```
.\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - C
.\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - C
```

This registers `Snowflake – Source` in the Data Map under the collection. **Pass/fail:** `Snowflake – Source` appears in the Data Map under `Snowflake – Collection` (portal / `DGSNOW-Discover.ps1`).

Step 4 — Create and run the scan on the pilot (Pilot)

1. **Create the scan** referencing the credential *by name* and the scan rule set, scoped to the pilot

database/schema:

```
.\DGSNOW-Register-Source.ps1 -Action CreateScan -PurviewAccount "<acct>" -SnowflakeServer "<account>.sn
.\DGSNOW-Register-Source.ps1 -Action CreateScan -PurviewAccount "<acct>" -SnowflakeServer "<account>.sn
```

1. **Trigger the scan** (portal or `-Action RunScan`) on the **pilot scope only**. Watch it complete; note

the assets discovered and the classifications applied. **Pass/fail:** the scan runs on the pilot database/schema and completes without auth/network errors; assets and classifications begin to appear. Discrepancies are recorded for troubleshooting, not worked around.

Step 5 — Validate the pilot (Pilot → Validate)

1. Run `DGSNOW-Validate.ps1 -PurviewAccount "<acct>"`. Read-only, it confirms `Snowflake – Source` is

registered under the collection, `Snowflake – Scan` exists, the last scan **completed**, and assets/ classifications are present for the pilot scope — PASS/FAIL per gate.

1. In the catalog, confirm the pilot database's tables/columns are discoverable and that expected

classifications (e.g. a PII column) applied.

1. Complete the **Testing Guide** on the pilot. **Pass/fail:** all Testing-Guide gates pass on

`Snowflake – Pilot Scope – Stage1`.

Step 6 — Expand and schedule (Expand)

1. Broaden `Snowflake – Scan` scope to the intended databases/schemas once the pilot validates, and set a

recurring schedule so the catalog stays current.

1. **Curate** — assign data owners/stewards, glossary terms, and review classifications with the

governance team (their determination, your enablement).

1. Run `DGSNOW-Report.ps1 -PurviewAccount "<acct>" -CsvPath .\SnowflakeReport.csv` to produce

`Snowflake – Coverage Report` — the source, scans, last-run status, and a classification summary — as the management evidence artifact, and re-run it on a schedule. **Pass/fail:** the scan covers the intended databases on a schedule; the report shows assets and classifications as designed.

Rollback

• **Disable / delete the scan** (portal or `-Action DeleteScan`) to stop scanning; the already-cataloged

metadata remains until you remove it.

- **Remove the registered source** to take Snowflake out of the Data Map (removes its assets from the catalog). Prefer disabling the scan over deleting the source while tuning.
- **Rotate the credential** in Key Vault if it is ever suspected exposed — the scan references the Key Vault secret, so rotating there updates the scan without touching any script. Keep `Snowflake Governance – Map` as the record of the intended state.

Order of operations

`Assess collections + runtimes → design collection/scope/rule set/credential (map) → prepare Key Vault-backed credential + register source → create + run scan on pilot db/schema → validate assets + classifications → expand scope + schedule + curate → report coverage`

5. Testing Guide — Snowflake Data Governance (Microsoft Purview)

Run after each configuration step. Every test is safe (read-only or a `-WhatIf` preview) except where it asks you to run a scan on the pilot scope. Validate on `Snowflake – Pilot Scope – Stage1` (one database/schema) before scanning the account.

Test 1 — Prerequisites and enablement are in place

Do: Run `DGSNOW-Check-Prereqs.ps1`. **Expected:** Az.Accounts present and connected; a Purview endpoint token can be acquired; the Purview account and Data-Map read signals reported; reminders to confirm the Key Vault linkage, SHIR, and roles. Every gap (e.g. SHIR not registered → network path) is **recorded, not guessed**.

Test 2 — Discovery inventory is complete

Do: Run `DGSNOW-Discover.ps1 -PurviewAccount "<acct>"`. **Expected:** The Data Map collections, existing registered sources and scans, and registered integration runtimes are listed — matching what the Purview portal shows.

Test 3 — Register-source dry-run makes no changes

Do: Run `DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - Collection" -SnowflakeServer "<account>.snowflakecomputing.com" -WhatIf`. **Expected:** It states the source it *would* register (name, collection, server) plus the reverse action and changes nothing; a follow-up `DGSNOW-Discover.ps1` shows no new source until `-Commit`.

Test 4 — Source registered under the collection

Do: After `-Commit`, run `DGSNOW-Discover.ps1` and check the Purview portal Data Map. **Expected:** `Snowflake – Source` appears in the Data Map under `Snowflake – Collection` with the correct Snowflake server.

Test 5 — Credential is Key Vault-backed, no secret in code

Do: Inspect the scan configuration and the scripts. **Expected:** The scan references `Snowflake – Scan Credential` **by name**; the secret lives in Azure Key Vault linked to Purview; **no password appears in any script, parameter, or document**. A secret in code is a fail.

Test 6 — Scan created on the pilot scope

Do: Run `DGSNOW-Register-Source.ps1 -Action CreateScan ... -PilotDatabase "PILOT_DB" -WhatIf` then `-Commit`. **Expected:** `-WhatIf` shows the scan it *would* create (credential-by-name, rule set, pilot database) and changes nothing; `-Commit` creates `Snowflake – Scan` scoped to the pilot database only.

Test 7 — Scan runs and completes on the pilot

Do: Trigger the scan on the pilot scope (portal or `-Action RunScan`) and watch its status. **Expected:** The scan connects to Snowflake (via the runtime), runs on the pilot database/schema, and completes with a **Succeeded** status — no authentication or network errors.

Test 8 — Assets appear in the catalog

Do: In the Purview catalog, search for the pilot database's tables. **Expected:** The scanned tables/columns are discoverable as assets under `Snowflake – Source`, with their technical metadata (schema, columns, data types).

Test 9 — Classifications applied where expected

Do: Open a table you know contains sensitive data (e.g. a column of emails or IDs) and review its classifications. **Expected:** The expected classifications / sensitive information types are applied to the sensitive columns per the scan rule set. (Verify the classification behavior against current Microsoft guidance.)

Test 10 — Validation gate passes on the pilot

Do: Run `DGSNOW-Validate.ps1 -PurviewAccount "<acct>"`. **Expected:** PASS for each gate — source registered under the collection, scan exists, last scan **completed**, and assets/classifications present for the pilot scope. Any FAIL names the specific issue.

Test 11 — Reporting evidence is produced

Do: Run `DGSNOW-Report.ps1 -PurviewAccount "<acct>" -CsvPath .\SnowflakeReport.csv`. **Expected:** `Snowflake – Coverage Report` lists the source, its scans, last-run status, and a classification summary — usable as the management evidence artifact and re-runnable on a schedule.

Pass/fail summary

All gates green on `Snowflake – Pilot Scope – Stage1` — source registered, scan completed on the pilot database, assets catalogued and sensitive columns classified, credential Key Vault-backed with no secret in code, and the coverage report produced — before expanding scope, scheduling, and curating. Any deferred (connector/runtime/licensing) constraint is recorded explicitly, not silently skipped.

6. Troubleshooting — Snowflake Data Governance (Microsoft Purview)

Symptoms, likely causes, and fixes for onboarding Snowflake to the Purview Data Map. Stay on the **pilot scope** (one database/schema) while diagnosing — prove the fix there before scanning the account. **Never** put a Snowflake secret in a script to "test" a connection; the secret lives only in Key Vault.

The scan fails to authenticate to Snowflake

• **Credential not linked / wrong reference.** The scan references the Purview **credential by name**,

which must point at the **Key Vault secret**. Confirm the Key Vault is linked to the Purview account (Management → Credentials) and the credential resolves the secret. Rotating/fixing the secret is a Key Vault action — no script change.

• **Snowflake login/role insufficient.** The scanning login needs read access to the in-scope databases'

metadata. Verify the Snowflake user/role and that it isn't locked/expired.

• **Wrong server endpoint.** Confirm the Snowflake server (`<account>.snowflakecomputing.com`) and any

required warehouse/role parameters match the connector's expectations.

The scan fails to connect (network)

• **No path from the runtime.** If Snowflake isn't reachable from the managed runtime, a **self-hosted

integration runtime (SHIR)** on a host with connectivity is required. `DGSNOW-Discover.ps1` lists registered runtimes; confirm one can reach the Snowflake host/port.

• **Firewall / Snowflake network policy.** Snowflake network policies or corporate firewalls may block the

runtime's IP. Open the path under change control and confirm with the Snowflake admin.

`DGSNOW-Register-Source.ps1 -Commit` returns a REST error

• **Token/permissions.** Ensure `Connect-AzAccount` succeeded and a Purview token was acquired

(`DGSNOW-Check-Prereqs.ps1`), and that the identity has **Data source admin** on the collection.

• **API version / schema drift.** The Data Map REST schema, connector `kind`, and API version **evolve**.

If the body is rejected, verify the current Snowflake connector request shape and `-ApiVersion` against Microsoft's documentation for your account, and adjust.

• **Collection name.** The `-Collection` reference must be the collection's **name** (not its friendly

name). Confirm with `DGSNOW-Discover.ps1`.

The scan runs but finds no assets

- **Scope too narrow / wrong database.** Confirm `-PilotDatabase` matches an existing database the login

can read. Widen within the pilot if the target schema is empty.

- **Still running.** Large scans take time. Check the run status (`DGSNOW-Validate.ps1` / portal) before

concluding it found nothing.

- **Permissions on objects.** The login may see the database but not specific schemas/tables. Grant the

least-privilege read it needs on the in-scope objects.

Classifications aren't applied to columns I expected

- **Scan rule set.** Classifications come from the scan rule set. Confirm `Snowflake – Scan Rule Set`

includes the classifications/SITs you expect, and that the column data actually matches them.

- **Sampling / thresholds.** Classification uses sampling and confidence thresholds; a column may not

classify if the sample didn't meet them. Verify the behavior against current Microsoft guidance before concluding it's wrong; re-scan if the rule set changed.

- **Custom classifications not defined.** If you expected a custom classification, confirm it's created in

Purview and included in the rule set.

Scans are consuming more capacity/cost than expected

- **Full-account scan too early.** Keep to the **pilot scope** until validated, then expand deliberately and

schedule rather than repeatedly full-scanning. Data Governance scans consume capacity — confirm the consumption model with the owner and right-size the schedule.

`DGSNOW-Validate.ps1` shows the run status as failed or empty

- **Read the run detail in the portal.** The REST status is a summary; open the scan run in the Purview

portal for the detailed error (auth, network, permissions) and fix per the sections above. Re-run on the pilot scope after the fix.

When to stop and escalate

- A connector/runtime/consumption constraint blocks progress on your account — record it **deferred**,

route the licensing/consumption decision to the owner, and confirm the current connector support with Microsoft; do not design around an unavailable capability.

- **The classification scheme, data ownership, and glossary** are data-governance determinations, not

technical fixes — capture them in `Snowflake Governance – Map` and defer the determinations.

- If a scan can't be made to complete on the pilot, gather `DGSNOW-Validate.ps1` output and the portal run

detail as evidence and raise with Microsoft support before expanding scope — never widen a scan that fails on one database.

7. Best Practices — Snowflake Data Governance (Microsoft Purview)

- **Confirm the connector and consumption model before you design anything.** Purview Data Governance is

evolving (classic Data Map and the newer unified catalog), the Snowflake connector's requirements change, and scans **consume capacity**. Read the current connector docs and confirm the pricing/consumption model for *your* account before scanning — surprises here are budget surprises.

- **Never put a Snowflake secret in a script.** Store the credential secret in **Azure Key Vault**, link

the Key Vault to Purview, and reference the Purview **credential by name**. The scripts pass the credential name, never a password. Rotating the secret is then a Key Vault action that touches no script.

- **Least-privilege the Snowflake scanning login.** The scan only needs to read metadata for the in-scope

databases. Give the Snowflake user/role the minimum read it needs — not an admin role — so a compromised scan credential is low-impact.

- **Pilot on one database/schema first.** A full-account scan is expensive and surfaces more findings than

you can curate at once. Prove the connection, scope, and classifications on `Snowflake – Pilot Scope – Stage1` before expanding.

- **Get the integration runtime right early.** If Snowflake isn't reachable from the managed runtime, a

self-hosted integration runtime (SHIR) on a host with connectivity is required. Sort the network path and SHIR before creating the scan — most scan failures are connectivity, not configuration.

- **The classification scheme and owners are governance determinations.** Which classifications matter, who

owns the data, and the glossary come from data-governance/privacy. Configure the scan rule set they specify and curate to their direction; capture determinations and stewards in `Snowflake Governance – Map`.

- **Schedule scans so the catalog stays current.** A one-time scan goes stale as Snowflake changes. Put

`Snowflake – Scan` on a recurring schedule so the catalog reflects the live estate, and re-report to catch drift.

- **Curate, don't just connect.** Registering and scanning is the foundation; the value comes from

assigning owners, glossary terms, and validating classifications. Treat curation as part of "done," led by the governance team.

- **Scope classifications deliberately.** Applying every classification to every column is noisy. Choose

the classifications/SITs that matter for Snowflake in the scan rule set so the findings are actionable, not overwhelming.

- **Keep automation safe-by-default and secret-free.** Run discovery, validation, and reporting read-only;

gate every write (register/scan) behind `-WhatIf` → `-Commit`; sign in interactively; reference the credential by name only. No secrets in a script or a document.

- **Keep determinations with their owners.** The solution registers, scans, and classifies Snowflake; the

classification scheme, data ownership, glossary, and any regulatory obligation are determinations for data-governance, compliance, and privacy. Keep that separation explicit so cataloging is never mistaken for a governance ruling.

8. Deliverables Summary — Snowflake Data Governance (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Registering, scanning, and classifying Snowflake in the Purview Data Map + catalog; Key Vault-backed credential, pilot-scoped scan; how cataloging differs from DLP (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: assess collections/runtimes, design the approach, prepare the Key Vault-backed credential + register the source, create and run the scan on a pilot database, validate, expand + schedule + curate
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Collection	`Snowflake – Collection`
Registered source	`Snowflake – Source`
Scan	`Snowflake – Scan`
Scan rule set	`Snowflake – Scan Rule Set`
Credential (Key Vault-backed)	`Snowflake – Scan Credential`
Pilot scope	`Snowflake – Pilot Scope – Stage1`
Design map	`Snowflake Governance – Map`
Coverage report	`Snowflake – Coverage Report`

PowerShell scripts

Script	What it does
`DGSNOW-Discover.ps1`	Reports current state: Data Map collections, existing registered sources and scans, and registered integration runtimes — PowerShell/REST against the Purview Data Map; read-only
`DGSNOW-Check-Prereqs.ps1`	Prerequisite/health helper: Az module + connection, the Purview-account and Data-Map read signals, and reminders to confirm the Key Vault linkage, SHIR, and roles — PASS/GAP per signal; read-only
`DGSNOW-Register-Source.ps1`	Registration/Scan: registers `Snowflake – Source` under the collection (`-Action RegisterSource`), creates `Snowflake – Scan` referencing the Key Vault-backed credential **by name** and scoped to the pilot database (`-Action CreateScan`), and triggers/deletes scans (`-Action RunScan/DeleteScan`); **defaults to** `-WhatIf` (prints the plan + reverse action), applies only with `-Commit`; **no secret in code**
`DGSNOW-Validate.ps1`	Validation: PASS/FAIL that the source is registered under the collection, the scan exists, the last scan completed, and assets/classifications are present for the pilot scope — read-only
`DGSNOW-Report.ps1`	Coverage report — the source, its scans, last-run status, and a classification summary — written to CSV as `Snowflake – Coverage Report`; read-only

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Snowflake governed from Purview — registered in the Data Map, scanned on a schedule, and its sensitive assets classified in the catalog — proven on a pilot database before scanning the account, with the credential Key Vault-backed (no secret in code) and the classification scheme, data ownership, and glossary left to the data-governance owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGSNOW-Check-Prereqs.ps1

Purpose: Confirms the environment is ready to register/scan Snowflake: the Az.Accounts module and Azure sign-in, whether a Purview endpoint token can be acquired and the Data Map is readable, and prints the manual checks (Key Vault-backed credential, SHIR, least-privilege roles). Read-only; stores no secret. HOW TO RUN .\DGSNOW-Check-Prereqs.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal plus a manual checklist. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Snowflake Data Governance. Read-only. Verify your licensing/tenant: Data Governance has its own consumption model; the Snowflake connector + API surface are evolving. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$ApiVersion = '2022-07-01-preview')

How to run: .\DGSNOW-Check-Prereqs.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal plus a manual checklist. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Snowflake Data Governance. Read-only. Verify your licensing/tenant: Data Governance has its own consumption model; the Snowflake connector + API surface are evolving. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$ApiVersion = '2022-07-01-preview')

DGSNOW-Discover.ps1

Purpose: Reports the current Data Map state for planning Snowflake onboarding: the collections, existing registered data sources and their scans, and any registered integration runtimes. Uses the Purview Data Map REST API with a bearer token acquired at runtime (Connect-AzAccount). Read-only — makes no change and stores no secret. HOW TO RUN .\DGSNOW-Discover.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount (modern-auth/MFA admin). Az.Accounts module. EXPECTED RESULTS A list of collections, sources+scans, and integration runtimes — captured into 'Snowflake Governance - Map'. Makes no configuration change. .NOTES Purview — Snowflake Data Governance. Read-only discovery. Verify your licensing/tenant: the Data Map REST surface + Snowflake connector are evolving; confirm the current API version and portal flow. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$ApiVersion = '2022-07-01-preview' # verify the current Data Map API version for your account)

How to run: .\DGSNOW-Discover.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount (modern-auth/MFA admin). Az.Accounts module. EXPECTED RESULTS A list of collections, sources+scans, and integration runtimes — captured into 'Snowflake Governance - Map'. Makes no configuration change. .NOTES Purview — Snowflake Data Governance. Read-only discovery. Verify your licensing/tenant: the Data Map REST surface + Snowflake connector are evolving; confirm the current API version and portal flow. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$ApiVersion = '2022-07-01-preview' # verify the current Data Map API version for your account)

DGSNOW-Register-Source.ps1

Purpose: Automates Snowflake onboarding SAFELY via the Purview Data Map REST API. -Action RegisterSource registers 'Snowflake - Source' under the collection. -Action CreateScan creates 'Snowflake - Scan' referencing the Key Vault-backed credential BY NAME and scoped to the pilot database. -Action RunScan triggers a run; -Action DeleteScan removes a scan. DEFAULTS TO -WhatIf: prints exactly what it would do (the REST call + reverse action) and changes nothing; applies only with -Commit. The Snowflake secret is NEVER passed here — it lives in Azure Key Vault, referenced by the Purview credential. HOW TO RUN .\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - Collection" -SnowflakeServer "<account>.snowflakecomputing.com" -WhatIf .\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - Collection" -SnowflakeServer "<account>.snowflakecomputing.com" -Commit .\DGSNOW-Register-Source.ps1 -Action CreateScan -PurviewAccount "<acct>" -SnowflakeServer "<account>.snowflakecomputing.com" -CredentialName "Snowflake - Scan Credential" -PilotDatabase "PILOT_DB" -Commit .\DGSNOW-Register-Source.ps1 -Action RunScan -PurviewAccount "<acct>" -Commit PowerShell 5.1+/7+. Connect first with Connect-AzAccount (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: the planned REST body + reverse action, no change. -Commit: the source/scan exists or the run is triggered (verify with DGSNOW-Discover.ps1 / the portal). No secret is stored or transmitted by this script. .NOTES Purview — Snowflake Data Governance. Safe-by-default; no secret in code. Verify your licensing/tenant: the Snowflake connector schema, scan-rule-set, and API version evolve — confirm the current REST shape. > [CmdletBinding()] param([ValidateSet('RegisterSource','CreateScan','RunScan','DeleteScan')] [string]$Action = 'RegisterSource', [Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ScanName = 'Snowflake - Scan', [string]$Collection, [string]$SnowflakeServer, # e.g. <account>.snowflakecomputing.com [string]$CredentialName = 'Snowflake - Scan Credential', # Purview credential (Key Vault-backed) — name only [string]$ScanRuleSet = 'Snowflake - Scan Rule Set', [string]$PilotDatabase, [string]$ApiVersion = '2022-07-01-preview', [switch]$WhatIf, [switch]$Commit)

How to run: .\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - Collection" -SnowflakeServer "<account>.snowflakecomputing.com" -WhatIf .\DGSNOW-Register-Source.ps1 -Action RegisterSource -PurviewAccount "<acct>" -Collection "Snowflake - Collection" -SnowflakeServer "<account>.snowflakecomputing.com" -Commit .\DGSNOW-Register-Source.ps1 -Action CreateScan -PurviewAccount "<acct>" -SnowflakeServer "<account>.snowflakecomputing.com" -CredentialName "Snowflake - Scan Credential" -PilotDatabase "PILOT_DB" -Commit .\DGSNOW-Register-Source.ps1 -Action RunScan -PurviewAccount "<acct>" -Commit PowerShell 5.1+/7+. Connect first with Connect-AzAccount (modern-auth/MFA admin). EXPECTED RESULTS -WhatIf: the planned REST body + reverse action, no change. -Commit: the source/scan exists or the run is triggered (verify with DGSNOW-Discover.ps1 / the portal). No secret is stored or transmitted by this script. .NOTES Purview — Snowflake Data Governance. Safe-by-default; no secret in code. Verify your licensing/tenant: the Snowflake connector schema, scan-rule-set, and API version evolve — confirm the current REST shape. > [CmdletBinding()] param([ValidateSet('RegisterSource','CreateScan','RunScan','DeleteScan')] [string]$Action = 'RegisterSource', [Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ScanName = 'Snowflake - Scan', [string]$Collection, [string]$SnowflakeServer, # e.g. <account>.snowflakecomputing.com [string]$CredentialName = 'Snowflake - Scan Credential', # Purview credential (Key Vault-backed) — name only [string]$ScanRuleSet = 'Snowflake - Scan Rule Set', [string]$PilotDatabase, [string]$ApiVersion = '2022-07-01-preview', [switch]$WhatIf, [switch]$Commit)

DGSNOW-Report.ps1

Purpose: Builds 'Snowflake - Coverage Report' — the registered Snowflake source, its scans, and each scan's most recent run status — as the management evidence artifact. Uses the Purview Data Map REST API with a token acquired at runtime. Read-only; stores no secret; writing a CSV is opt-in. HOW TO RUN .\DGSNOW-Report.ps1 -PurviewAccount "<your-purview-account>" .\DGSNOW-Report.ps1 -PurviewAccount "<your-purview-account>" -CsvPath .\SnowflakeReport.csv PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of the source, its scans, and last-run status — re-runnable on a schedule. (Deep classification counts are reviewed in the catalog UI.) Makes no configuration change. .NOTES Purview — Snowflake Data Governance. Read-only reporting. Verify your licensing/tenant: the run-status schema + API version evolve. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ApiVersion = '2022-07-01-preview', [string]$CsvPath)

How to run: .\DGSNOW-Report.ps1 -PurviewAccount "<your-purview-account>" .\DGSNOW-Report.ps1 -PurviewAccount "<your-purview-account>" -CsvPath .\SnowflakeReport.csv PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of the source, its scans, and last-run status — re-runnable on a schedule. (Deep classification counts are reviewed in the catalog UI.) Makes no configuration change. .NOTES Purview — Snowflake Data Governance. Read-only reporting. Verify your licensing/tenant: the run-status schema + API version evolve. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ApiVersion = '2022-07-01-preview', [string]$CsvPath)

DGSNOW-Validate.ps1

Purpose: The pilot/expansion gate. Confirms PASS/FAIL that 'Snowflake - Source' is registered, 'Snowflake - Scan' exists, and the most recent scan run completed successfully — via the Purview Data Map REST API with a token acquired at runtime. Read-only; stores no secret. Run on the pilot before expanding. HOW TO RUN .\DGSNOW-Validate.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; overall PASS only when the source + scan exist and the last run succeeded. Any FAIL names the specific missing object or the failed run status. (Confirm classifications in the catalog UI — the deep asset/classification check is done in the portal.) .NOTES Purview — Snowflake Data Governance. Read-only gate. Verify your licensing/tenant: the scan-run status schema + API version evolve; confirm against your account. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ScanName = 'Snowflake - Scan', [string]$ApiVersion = '2022-07-01-preview')

How to run: .\DGSNOW-Validate.ps1 -PurviewAccount "<your-purview-account>" PowerShell 5.1+/7+. Connect first with Connect-AzAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per gate; overall PASS only when the source + scan exist and the last run succeeded. Any FAIL names the specific missing object or the failed run status. (Confirm classifications in the catalog UI — the deep asset/classification check is done in the portal.) .NOTES Purview — Snowflake Data Governance. Read-only gate. Verify your licensing/tenant: the scan-run status schema + API version evolve; confirm against your account. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PurviewAccount, [string]$SourceName = 'Snowflake - Source', [string]$ScanName = 'Snowflake - Scan', [string]$ApiVersion = '2022-07-01-preview')